

ALUConnect

Bridging ALU Students and Student-Led Startups
Through a Mobile Internship Marketplace

Prepared by: ISHIMWE Yvan Ralph
Individual Assignment 2 (Flutter Project)
July 2026

Abstract

The ALU students are facing difficulties in getting practical and relevant experience because all internship portals are created to create a bridge between external companies and interns whereas the start-ups at ALU do not even consider the possibility of utilizing the potential students of their own university for development, design, marketing, operations, research, and content purposes. The application is made in Flutter language and is named ALUConnect, in which there is a list of start-ups of ALU and students can look for their desired internships through this portal. This is done through Firebase Authentication and Cloud Firestore.

This document encompasses a brief description on architectural design, Firestore data modeling, state management with Provider / ChangeNotifier, key workflows, rationale for the choice of UI/UX and gate verifications, scalability and security approach, testing strategy, and main problems that occurred during the development process.

1. Introduction

1.1 Problem Statement

There are two types of ALU programs that face a common problem that cannot be solved by any currently existing resource. Internships are very valuable opportunities that provide great career experience. However, the currently existing internships cater to large outside firms with semester-unfriendly schedules that assume prior experience which is rarely owned by undergraduate students. At the same time, there is a lack of implementation capacity among startups established by ALU students. They require help with development, design, marketing, operations, research, and content production, but they do not have a way to reach out to students present on campus.

1.2 Project Overview

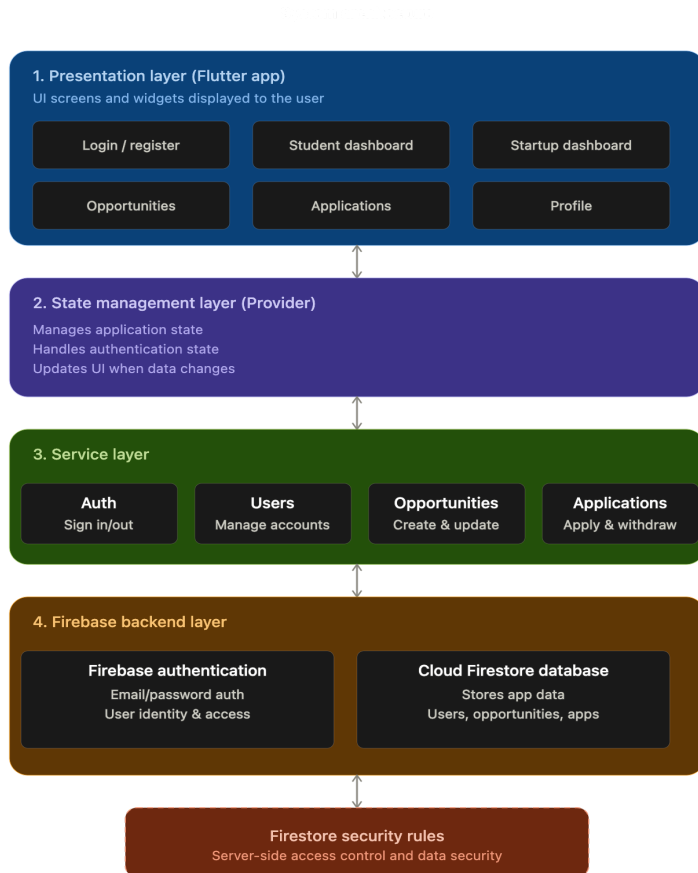
ALUConnect is a Flutter application that consists of two types of user profiles namely Student and Startup. These two types of users have a unique onboard process, home screen, and bottom navigation shell. For a startup to be verified, they should provide evidence of their connection with ALU (African Leadership University, n.d.), and this is possible only through verification by the admin through both UI and backend security. Only those startups that are verified are allowed to post their opportunities. The students are allowed to search for the available opportunities and see the information about it. They can also apply to it with just a click.

1.3 Objectives

- Create an installable application with Flutter framework running on an emulator or device.
- Utilize Firebase (Authentication & Cloud Firestore) as the backend for your project.
- Use state management (Provider/ChangeNotifier) in an efficient manner across all the screens.
- Include the basic set of features, along with some additional features like reviewing of applicants, profile, portfolio, accepting or rejecting of users, and even analytics bar.
- Include the necessary documentation which will justify the decisions you made in your project.

2. System Architecture

The architecture of ALUConnect is built using a layered structure comprising the following layers: Presentation (Flutter widgets/screens) -> State Management (Provider) -> Services Layer (one class per Firestore collection) -> Backend (Firebase Auth + Cloud Firestore with Security Rules).



2.1 Why a Layered Architecture

The entire application structure strictly relies on a unidirectional dependency relationship chain: Screens rely on services that then rely on the Firebase SDK, and the opposite is not permitted. The Screens do not have interaction with `FirebaseFirestore.instance` as all operations with read/write are made via a service class (Auth Services, Firestore Service, Opportunity Service, Application Services). This ensures that the schema of the fields in each collection resides in one place.

2.2 Folder Structure

lib/ Mostly organized according to their function.
 main.dart (firebase setup, multi-provider, theme)
 models/ (Plain Dart Data Class)
 providers/ (AuthProvider)
 services/ (Firestore Read and Write Service)
 screens/ (auth, home, opportunities, applicants, profile)
 widgets/ (UI Components)
 utils/ (Colors, Categories, Validation, Formatters)

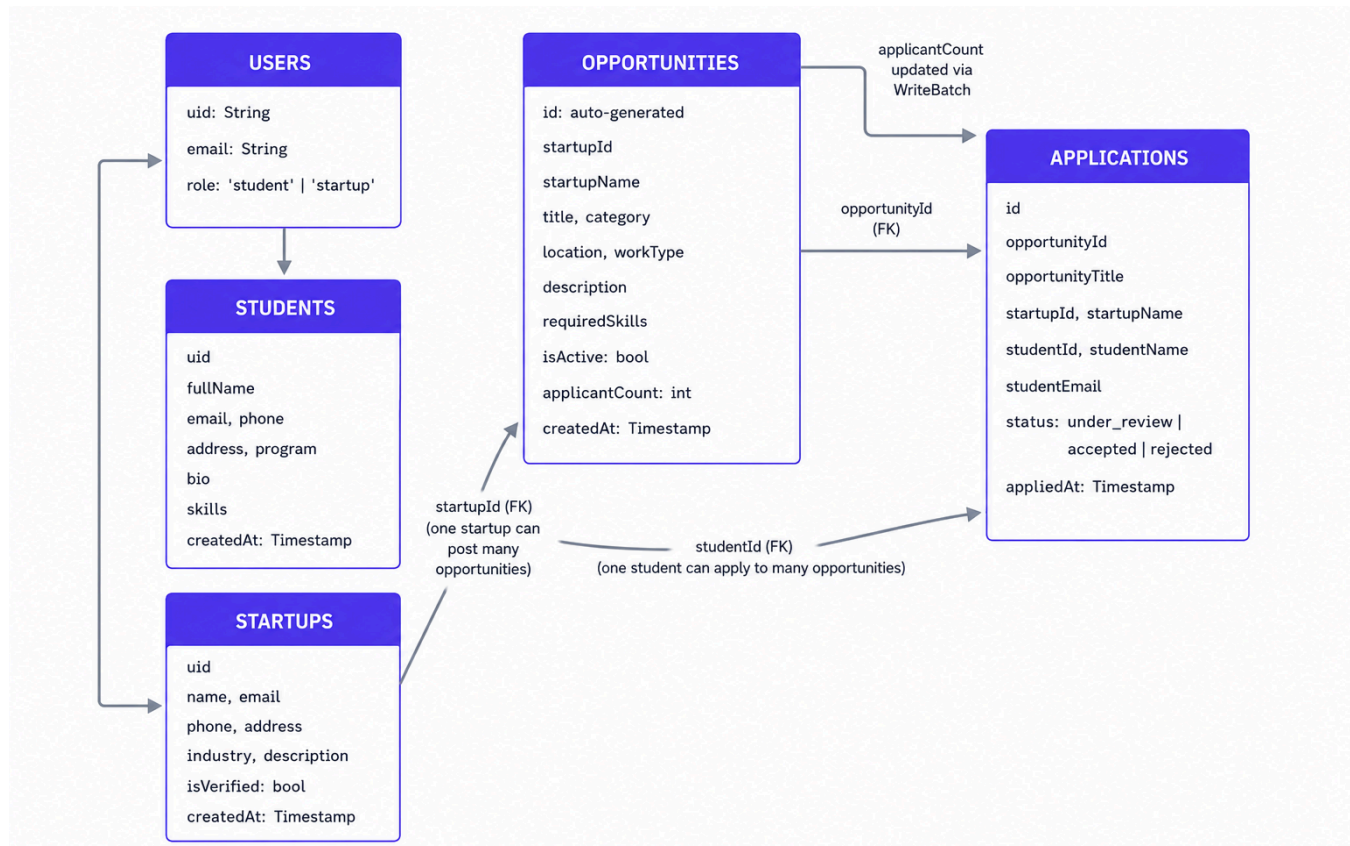
2.3 Navigation & Technology Choice

There is just the Navigator imperative stack within the navigation structure, with no named route table and no deep linking. It will be just the AuthWrapper that will check the AuthProvider to navigate among the Login, StudentHome, and the StartupHome screens. The homepage of each user role will have an IndexedStack with four tabs.

Flutter is the technology that was forced upon us in our assignment (Flutter, 2024). Flutter allows us to create a program using one codebase for both Android and iOS operating systems, and the same rendering engine. Firebase saves us from the trouble of creating our own back-end server; however, it still requires proper back-end design, along with Security Rules, schema, and index queries.

3. Firebase Backend Structure and Data Model

ALUConnect uses Cloud Firestore across five top-level collections: users, students, startups, opportunities, and applications (Firebase, 2024a).



3.1 Why Firestore, and How Relationships Are Modeled

The concept of joining is not available in Firestore. There are two ways through which ALUConnect circumvents this limitation: by using deterministic document IDs and by explicitly denormalizing. This is done by having the document ID of all applications be {opportunityId}_{studentId} as opposed to randomly generated, thereby ensuring that applying again is not logical since the second use of apply() would simply overwrite the initial document. Opportunities have their own startupName field, while applications have their own opportunityTitle, startupName, studentName, and studentEmail fields.

3.2 Collection Schemas (Summary)

Field	Type	Notes
users/{uid}	uid, email, role	Minimal account record; role is 'student' or 'startup', used to route to the correct home screen
students/{uid}	fullName, phone, address, program, bio, skills[], createdAt	Core profile; bio/skills optional, filled in via Profile Edit
startups/{uid}	name, phone, address, industry, description, aluAffiliationProof, isVerified, createdAt	isVerified defaults false; changed only via Firebase console by an admin, never by the app
opportunities/{id}	startupId, startupName, title, category, location, workType, description, requiredSkills[], isActive, applicantCount, createdAt	applicantCount kept in sync via FieldValue.increment inside a WriteBatch
applications/{oppId}_{studentId}	opportunityId, opportunityTitle, startupId, startupName, studentId, studentName, studentEmail, status, appliedAt	status: under_review accepted rejected

3.3 Query Design and Security Enforcement

Firestore needs a manual composite index when the query has `where()` along with `orderBy()` for another field; this is an often overlooked requirement that will silently make the app unusable for anyone using a new Firebase project, even a grader. The `OpportunityService` and `ApplicationService` classes do not run into this problem because they only use a `where()` query and sort locally afterwards in Dart.

This will ensure that only ALU startups that have been validated can post opportunities. This will be done twice using `ALUConnect` by first disabling the “post opportunity” button on the client side using the shared `requireVerifiedStartup()` gate and then again by checking the create firestore rule `startups/{uid}.isVerified == true`. This is because verification on the client side is not very secure. Also, only the applicant can withdraw whereas only the startup can modify the application status.

4. State Management Approach

In `ALUConnect`, `Provider` is used for implementing the `ChangeNotifier` approach for the global session state while `StreamBuilder` is employed to manage the local state of live data on individual screens.

4.1 Why Provider

`Provider` was chosen in comparison to `BLoC/Cubit` and `Riverpod` because it is the recommended option, and also the easiest to implement within Flutter (Lenz, 2024), in keeping with the goal of being beginner-friendly in this project that involves only one developer working on application with sessions and roles. `BLoC` and `Riverpod` both offer viable options for developing an application in larger teams.

4.2 AuthProvider and the watch/read Idiom

`AuthProvider` extends `ChangeNotifier` and consists of four properties, which are `authStatus` (unknown/authorized/unauthorized), `appUser` (uid, email, role), `isLoading`, and `errorMessage`. `AuthProvider` listens to the stream provided by Firebase on `authStateChanges` that are stored persistently, and once the sign-in is done successfully, it gets the role of the user through `users/{uid}`.

`context.watch<AuthProvider>()` is used only where a widget's build output must visibly react to a change (`AuthWrapper`, home shells, auth screens' submit buttons). Everywhere else — reading a uid in `initState`, or calling `logout()` from a button — uses `context.read<AuthProvider>()`, which does not subscribe to future changes. This distinction matters structurally, since `context.watch` cannot be used inside `initState` at all.

4.3 Local State and Optimistic Updates

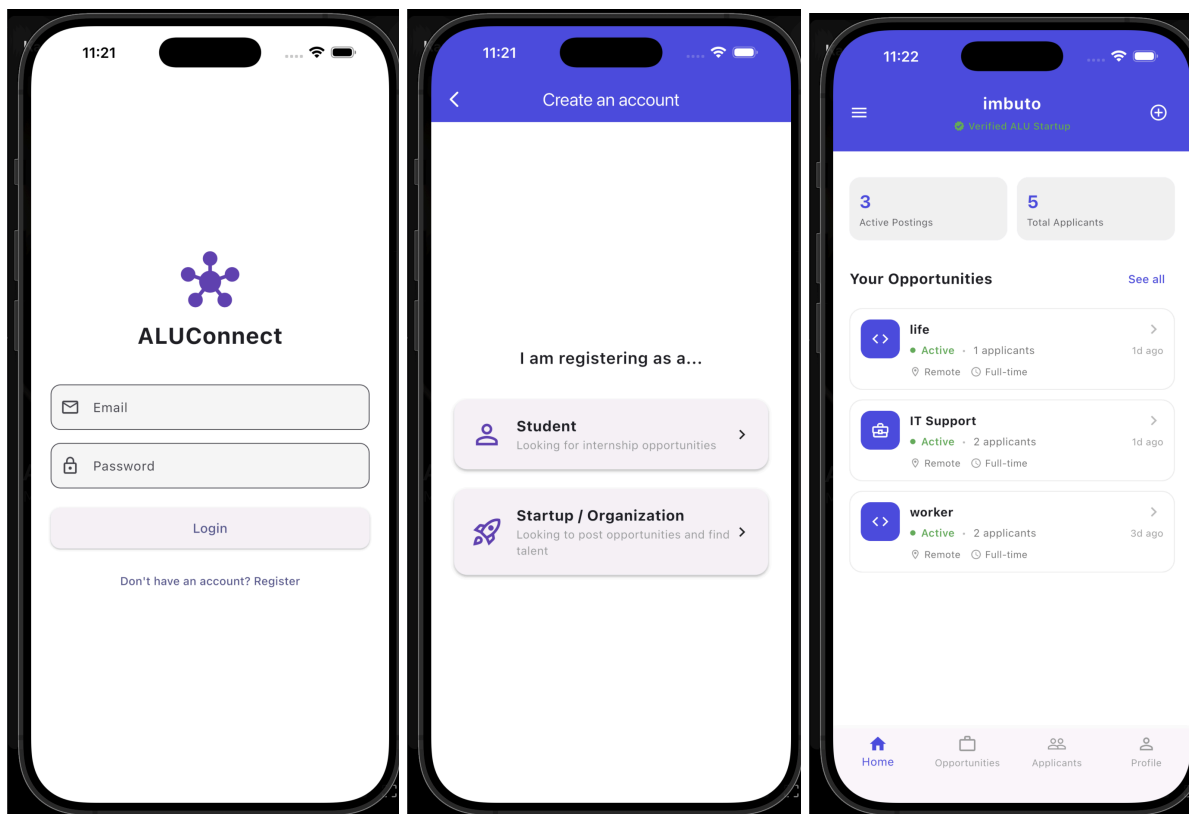
Every single list and details screen that uses data from Firestore is connected to StreamBuilder through a stream with `.snapshots()`, which is instantiated just once per widget lifetime and will be disposed of automatically – completely independent of AuthProvider as this stream is used by no other component besides the particular screen itself. There is only one screen that presents a third special case where a locally stored Boolean flag is set through `setState()` right after successfully completing the `apply()` write process, turning the "Apply now" button into "You applied".

5. Application Workflows

5.1 Authentication, Onboarding, and Verification

This registration is done in three steps, which include creating an account using Firebase Auth (Firebase, 2024c), saving at least the AppUser document in `users/{uid}`, and finally saving the complete profile in either `students/{uid}` or `startups/{uid}`. During registration, the new user can decide whether he wants to register himself as a Student or Startup and fill the respective form on RegisterRoleScreen. For the startups, the field `isVerified` is always false, since it cannot be verified independently.

It is unnecessary to gate the process of registration; you just need to register and prove your affiliation to ALU, and the company will be regarded as being unverified. In the tab Startup Home, there should either be a green badge that says "Verified" or an orange one that reads "Verification pending". "+" post-opportunity action will be gated by the `requireVerifiedStartup()` function, that is why the block rather than the form is mentioned here.

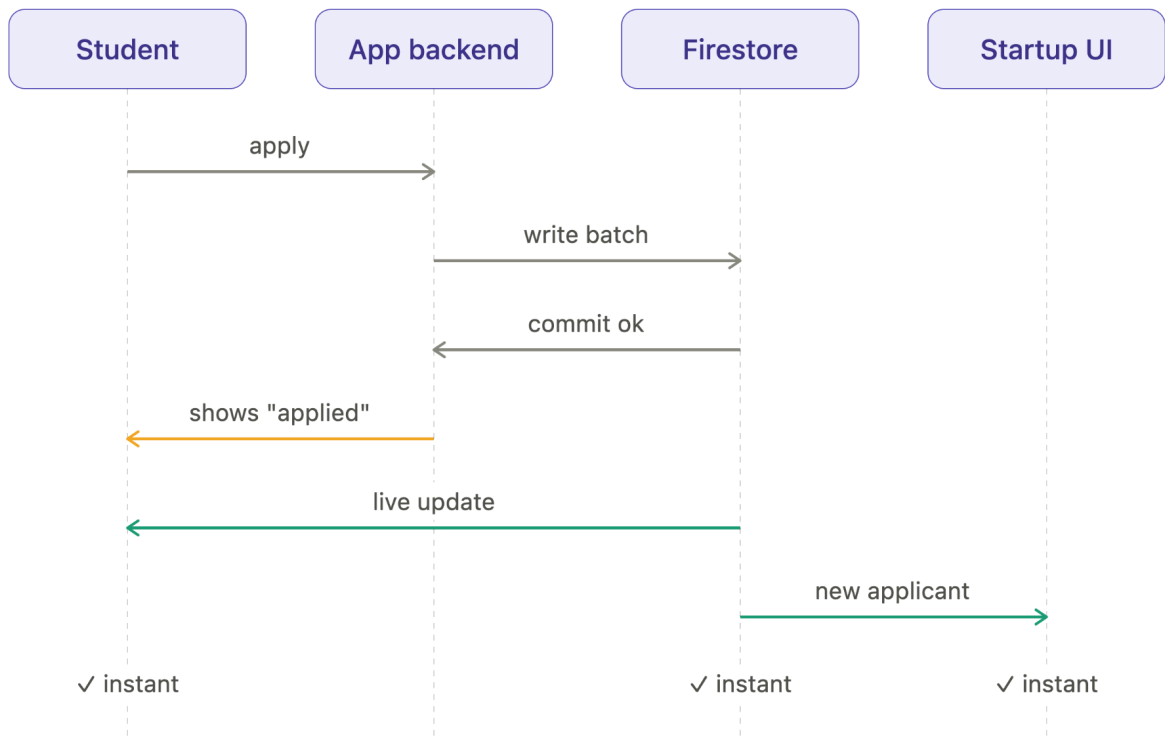


5.2 Posting, Discovery, and Applying

The authenticated startup submits using one consolidated form (OpportunityFormScreen), which takes care of edit and delete operations. The mandatory inputs include the title, category, location, and work type; the description and

skills are optional free inputs not to make it too complex by creating a chip input feature. Upon saving, the service always sets `isActive` to true, `applicantCount` to 0, and server timestamp.

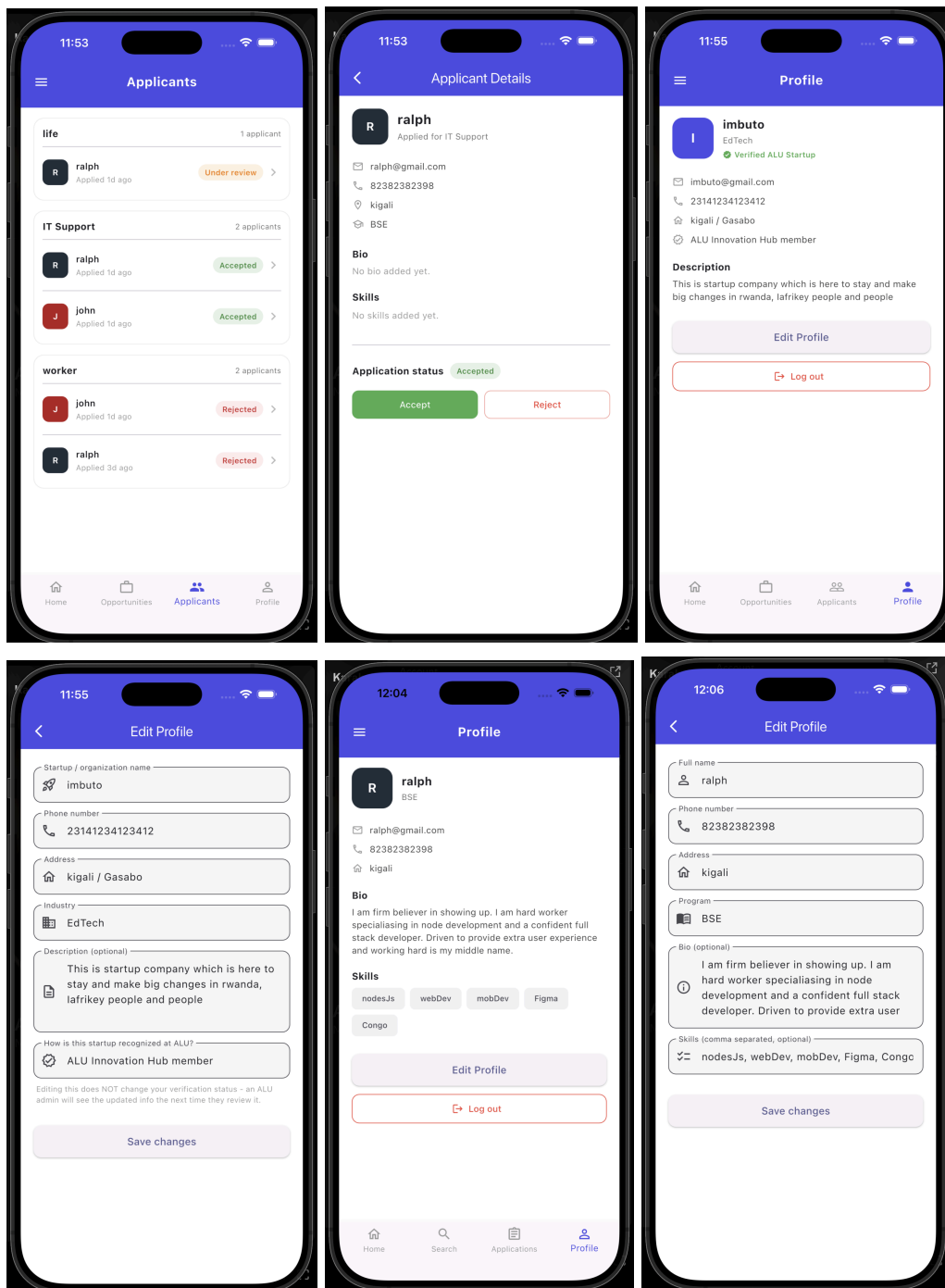
Opportunity is discovered by Student through the Home Tab (with filtering options based on categories) and Search Tab (with filtering options based on search). Clicking on the card redirects to the detail screen that features the Apply/Withdraw section live. Applying is accomplished in just a click, which does not require filling up of any form because `ApplicationService.apply()` generates an application record and increases `applicantCount` atomically through Firestore `WriteBatch` (Firebase, 2024d).



5.3 Reviewing Applicants and Profile Management

Applicants that have been applied to the Startup appear on the screen called Applicants. The screen displays all applicants who have applied to the Startup through its all postings. The applicants are organized according to opportunities. Upon tapping on an applicant’s name, one can see another screen displaying applicant's application, his/her profile and options like Accept/Reject. The process is reversible since both the buttons are tappable regardless of previous action.

Symmetric View+Edit exists for both roles. Email (associated with Firebase Auth) and, for start-ups, `isVerified` are never shown in edit mode and will always be passed through when saving edits to any profile.



6. UI/UX Design Reasoning

- Single source of truth branding: The whole design has one single value (AppColors.primary, which is indigo #4F46E5). Everything else is built around it (theme, app bars, common Header widget). It makes a brand change one line of code.
- AppAppBar: A reusable component that appears on all eight tabs instead of eight ad hoc headers. It can have an optional title, subtitle and trailing action in every tab.

- Frictionless forms: A minimum set of four fields (title, category, location, and type of work) for posting, and just tap-to-apply (on purpose, no CRUD-ish form).
- Statuses communication with colors and copy: Reusable StatusBadge and VerificationBadge components guarantee consistency in wording and colors when there's any kind of status to be shown.
- Clear states of loading, disabled and empty: All buttons have their labels turned into spinners depending on isLoading flag. Streams use connectionState to conditionally select branches in order to prevent an empty state from appearing, in line with established usability heuristics for visibility of system status (Nielsen, 1994).

7. Scalability and Security

7.1 Scalability

- The maximum number of where() filters per request is only one since the results are sorted client-side to avoid creating a composite index.
- Fields like startupName, studentName, etc., are denormalized and help avoid N+1 calls; however, they can't be updated if the source profile is updated after the fact.
- FieldValue.increment(), coupled with WriteBatch, guarantee atomicity and consistency of applicantCount in a concurrent application scenario.
- Since stateless service layer objects don't hold any user-related information, there are no caching or coordination mechanisms needed to scale horizontally.
- IndexedStack guarantees that all Firestore listeners of all four tabs work regardless of the tab you switch to by accepting small memory overhead cost.

7.2 Security

The firestore.rules file is the actual access control barrier for Firestore and not the Flutter application client itself (Firebase, 2024b). This is due to the fact that all the restrictions at the user interface level are usability related restrictions, but the firestore.rules file prevents any unauthorized reading and writing operations regardless of whether there is an attempt to bypass the application. All the collections of applications cannot be accessed by anyone anonymously as every rule starts from request.auth != null. Each application can be accessed only by the applicant who created it or the start-up that created it, as these applications contain information that should not be available to all the signed-in users. The limitation associated with this is that there is no validation that the application document creation or deletion happens in the ± 1 change rule of the applicantCount.

8. Testing Strategy

The testing process was primarily done manually in workflow-wise format keeping the scope of the project in mind. All modules were tested comprehensively in the real Firebase project and after that, another module was picked up in the following order: Authentication -> Role-Based Home screens -> Opportunity CRUD -> Apply -> View Applicants -> Profile -> UI.

- Every scenario was carried out by hand against the real Firebase console, making sure that the state of the UI matches the state of the document.
- Any changes to Security Rules were verified by performing the exact same thing that the rule was meant to disallow or allow for example, making sure that the request to create the entry comes back denied from the server, not just hidden from the client-side.

- Real-time capabilities were tested by having two users logged in at once (student and startup), and verifying that anything done on either end would show up automatically on the other.
- No static analysis was used, only manual code review (correct imports, null safety, controller disposal, etc.)

This method was able to identify issues in advance before implementation, such as the button being operable prior to loading of the student's profile, initial applicationCount being susceptible to change with any number rather than only plus 1 or minus, and title being unable to align itself centrally in the common header widget. No automated test has been included in this project; workflow testing was done in this project, whereas automated testing can be considered for the future.

9. Challenges, Lessons Learned, and Limitations

9.1 Key Challenges

- The rules for Firestore are not auto-deployed, and any modifications done in `firestore.rules` require manual publishing in the Firebase Console. Two permission denied exceptions were caused by this, and one of them could be resolved by including "did I publish the rules?" in the checklist, along with logging the exception raised by the firestore in every write operation's catch statement.
- Despite proper wiring with the StreamBuilder, the 'Apply' button failed to refresh its status because of a delay from the local Firestore cache that cannot guarantee to complete before the next paint operation. The resolution of this problem was achieved with the use of an `_optimistic` flag, named `_justApplied`, which would turn true right after the successful completion of any write operation.
- The centering functionality for the title in a shared header widget stopped working for tabs where there was no icon present on the right part of the screen, as the calculation was based on the remaining space instead of the total space. The solution of the problem included the redesign of the entire widget using Stack component in which the title would be positioned separately, forcing another widget to be moved below the title.

9.2 Lessons Learned

- The security rules are code that has to be changed with every single new collection and workflow "works locally" isn't enough evidence that an update was made; only the state on the console is what counts here.
- Correctly implemented StreamBuilder doesn't mean instant updates of UI; setting the optimistic flag is just right for the most important operation in the app.
- Components that can be reused have to be tested in the barebone conditions – the bug in the header appeared only if there was minimal data in the tab.
- Consistency of architecture (one service for each collection) proves to be very rewarding – all features built after the CRUD for Opportunities utilized existing streams and patterns.

9.3 Limitations

- Automated testing is not available now. All verifications were performed manually according to the processes.
- The applicantCount rule uses an increment/decrement by 1 without checking the existence of the corresponding application document.
- All start-up verifications were made manually through the Firebase console. There's no admin dashboard in the app.
- Fields for denormalization aren't updated automatically in case of the name change in the source profile.
- Push notifications don't exist. The real-time function operates when you open the screen.

- Search is a client-side substring search and doesn't include full-text/fuzzy search.

10. Future Improvements

- Increment validation of the applicantCount variable and the corresponding application update as a single server-side validation process.
- Notification (Firebase Cloud Messaging) in case of any change in the application status or any new application.
- App startup verification process to avoid manual console verification through an admin verification screen.
- Automated widget testing and unit test of the important process through the Firebase emulator.
- Matching/Suggestions of skills based on the information available in the schema.
- In-app messaging feature for a startup with an accepted applicant.
- Text search index for opportunities as the number of opportunities keeps growing (Algolia/Typesense).

11. Conclusion

ALUConnect architecture gives a full working solution for a certain issue that arises in ALU architecture and concerns having a communication line between the startups that were created by students and the skilled students from the institute. As we can see from the architecture, several architectural patterns have been used during the development of this application, and they include such as single service class per each collection usage, Provider for handling session state, StreamBuilder for getting live local data, usage of deterministic document ids, intentional denormalization, atomic WriteBatch operations for implementing required functionality and others, without introducing any unnecessary complexity.

References

- African Leadership University. (n.d.). *About ALU*. <https://www.alueducation.com>
- Firebase. (2024a). *Cloud Firestore documentation*. Google. <https://firebase.google.com/docs/firestore>
- Firebase. (2024b). *Cloud Firestore Security Rules*. Google. <https://firebase.google.com/docs/firestore/security>
- Firebase. (2024c). *Firebase Authentication documentation*. Google. <https://firebase.google.com/docs/auth>
- Firebase. (2024d). *Firestore transactions and batched writes*. Google. <https://firebase.google.com/docs/firestore/manage-data/transactions>
- Flutter. (2024). *Flutter documentation*. Google. <https://docs.flutter.dev>
- Lenz, R. (2024). *Provider package for Flutter (Version 6.1.2)*. Dart Packages. <https://pub.dev/packages/provider>
- Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>